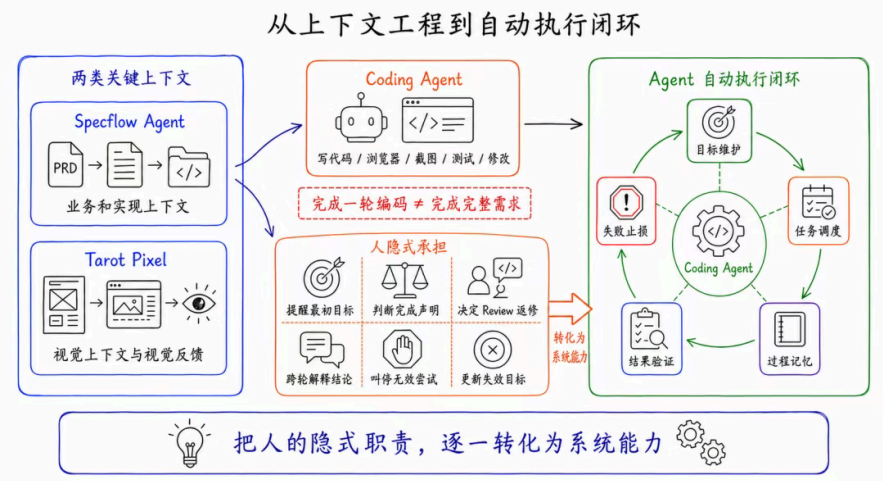
# 场景营销互动&体验 AI Coding — 构建 Agent 自主执行闭环

原创 营销&交易技术 大淘宝技术 2026年8月25日 19:04 浙江 11人

本文指出仅靠上下文工程无法保证长链路任务的成功，核心在于将目标维护、状态调度、结果验证等隐性人力工作转化为系统化的“循环工程”。通过引入结构化 Task 作为持久化目标锚点，定义 Job 为最小执行单元并强制要求结构化结果与证据，建立包含事实与判断的执行账本以实现状态外部化，以及基于工程责任划分多 Agent 角色并规范协作协议，系统能够有效处理执行偏差。此外，结合 Tarot Pixel 形成的视觉修正小循环，塔罗平台实现了从单次编码到全链路自动执行的跨越，最终目标是达成输入需求即可交付可发布代码的端到端研发闭环，让人类开发者仅专注于目标确认与异常仲裁。

## 01 前言




本文是场景营销互动&体验团队 AI Coding 系列文章的第三篇，前 2 篇文章：

- 场景营销前端 AI Coding — 从问题到方案：讲了塔罗 Specflow Agent 负责分析 PRD、设计稿和代码库，再把复杂需求整理为 Coding Agent 可以执行的任务。
- 场景营销前端 AI Coding — AI Native 的视觉稿还原：介绍了 Tarot Pixel — 一个把设计稿转化为 Coding Agent 可以持续查询、截图、比对和修正的视觉上下文的工具。

至此，形成了 AI Coding 所需的两类关键上下文：

- Specflow Agent（任务规划阶段） 提供**业务和实现上下文**；

- Tarot Pixel（编码阶段） 提供**视觉上下文**和视觉反馈。

业务上下文、视觉上下文和相关工具具备以后，执行编码似乎成为最后一个环节。然而，进入代码执行阶段后，一个新的问题逐渐显现：**Coding Agent 能完成一轮编码任务，不等于能完成一个完整需求**。

Coding Agent 可以写代码、打开浏览器、截图、跑测试，也可以根据一次反馈继续修改，但在一段更长的执行过程中，仍然需要人不断做这些事情：

- 提醒它不要忘记最初的目标；
- 判断 Agent 的完成声明是否成立；
- 决定 Review 打回以后应该由谁返修；
- 把上一轮的结论重新解释给下一轮；
- 发现它正在重复无效尝试，并及时叫停；
- 在需求变化时告诉它旧目标已经失效。

虽然具体操作由 Agent 执行，但人仍然承担着调度、状态维护和验收职责，这构成了塔罗当前阶段需要解决的核心问题。

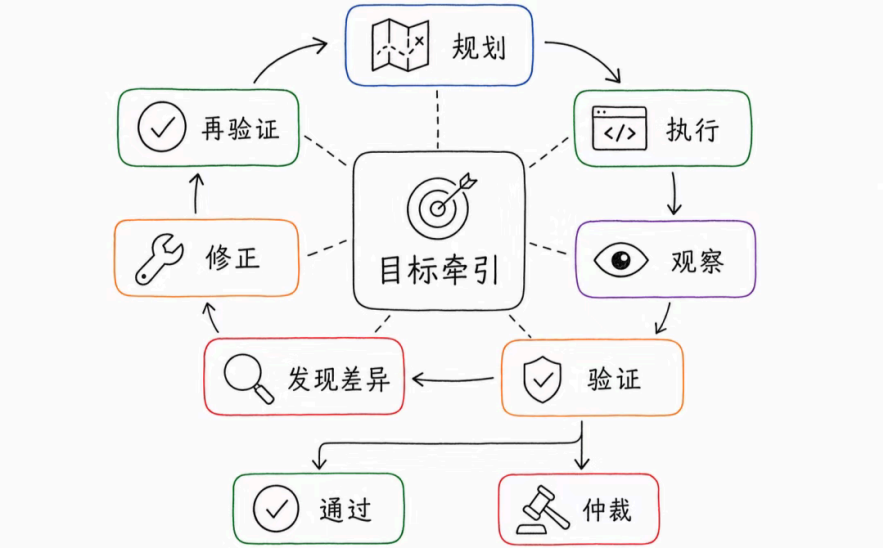
**构建 Agent 自动执行闭环的核心工作，是把过去由人隐式承担的目标维护、任务调度、过程记忆、结果验证和失败止损，逐一转化为系统能力。**

## 02 上下文工程提供执行输入，循环工程处理执行偏差

上下文只能提高单轮执行正确的概率，无法保证复杂任务一次成功。

真实业务开发天然包含不确定性：需求可能有歧义，代码入口可能找错，接口状态可能复现不了，UI 可能实现但没有精准还原设计稿，测试可能被判断通过但用户路径不成立，只要任务足够复杂，执行中出现错误就是常态。

自动执行需要建立对执行偏差的处理能力：即使单轮执行产生偏差，系统仍然能够识别目标、偏差、后续责任角色，以及必须转入人工处理的时机。




上下文、工具和循环作用于不同层面，上下文为 Agent 提供目标与约束，工具将执行过程连接到真实环境，循环则把本轮结果转化为下一步操作。前两者影响单轮执行质量，后者决定任务在出现偏差后能否继续推进。任一一层不足，目标的解释、结果的判断或后续调度都会重新落到开发者身上。

实现需求的自动执行闭环中暴露的多数问题，最终都指向同一个原因：**循环尚未被工程化**。

1. **目标缺乏持久化载体**

   “实现某某需求” 可以触发一次 Agent 执行，**但不足以支撑持续数十分钟并经历多轮返修的执行系统**。

   随着执行推进，初始目标会被中间过程逐渐稀释。Agent 可能只围绕最近一条 Review 意见工作，忽略任务的整体交付要求；也可能在解决局部问题后，将局部完成误判为任务完成。

2. **执行状态依赖对话历史**

   如果改动文件、已验证状态、截图对应的 Mock、Review 打回原因全部依赖对话历史，下一轮 Agent 就必须重新读取并推断执行状态。长对话成本较高且可靠性有限。对话内容同时包含事实、推测、计划和已失效结论，系统难以识别当前仍然有效的状态。

3. **完成声明缺乏验证**

   Coding Agent 在一次执行结束时可能给出如下结论："已完成 xx 功能，建议下一步进行 xx "，但对于自动执行系统而言，这一结论不足以驱动后续流程，系统无法从这句话中确认目标完成情况、验证证据和进入下一阶段，因此不能据此进行流程路由。

4. **反馈缺少责任归属**

   Review 发现“页面状态不完整”时，需要进一步区分视觉问题、前端状态问题、接口问题和产品边界问题。如果所有反馈都重新交给同一个通用 Agent，修改往往停留在表层，无法处理真正的责任边界。

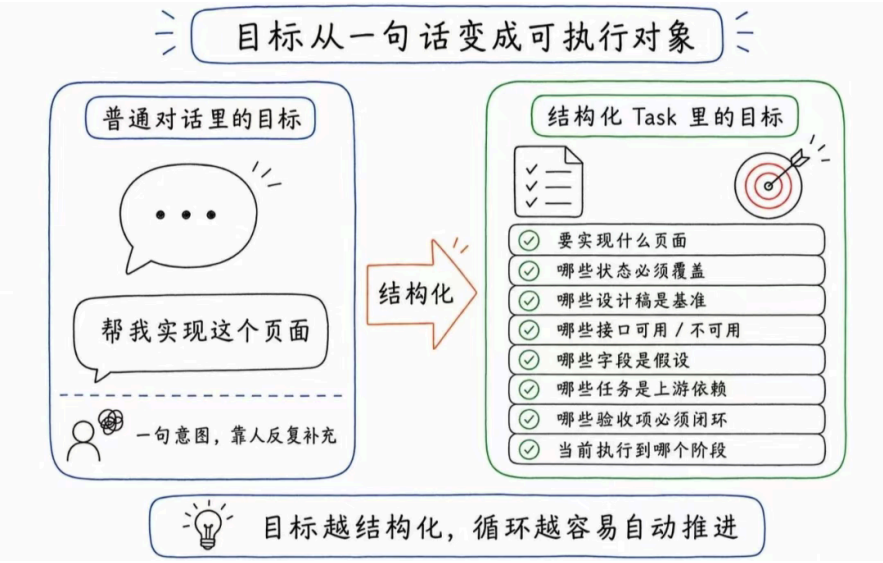
5. **循环缺乏收敛机制**

持续将失败结果作为下一轮输入，同样可以形成循环，但该循环可能只是在重复读取、解释和尝试，最终以不同措辞报告同一个失败。

循环工程关注的是目标、状态、证据、路由和停止条件如何共同构成一个可收敛的执行系统。

## 03 将目标转化为系统对象

塔罗第一版的 Specflow 会经过深度分析，生成任务文档。它解决了上下文准备的问题，但文档本身不会推动执行。塔罗第二版做的第一个变化，是把规划结果从一份“给人或 Agent 阅读的报告”，变成平台里的结构化 Task。




一个 Task 需要以结构化方式记录：

- 最终要达成什么结果；
- 哪些状态和用户路径属于本次范围；
- 哪些内容明确不做；
- 依赖哪些上游条件；
- 什么证据可以证明任务完成；
- 当前执行到了哪个阶段。




服务端需求任务面板

其作用是为整个执行过程提供稳定的目标锚点。

Agent 运行到第 5 分钟和第 50 分钟时，看到的对话上下文可能已经完全不同，但 Task 不应该跟着变化。除非用户明确修改目标，否则它始终是系统判断“有没有做完”的依据。

在塔罗中，任务被建模为可以保存、修改、验证和回溯的系统对象。

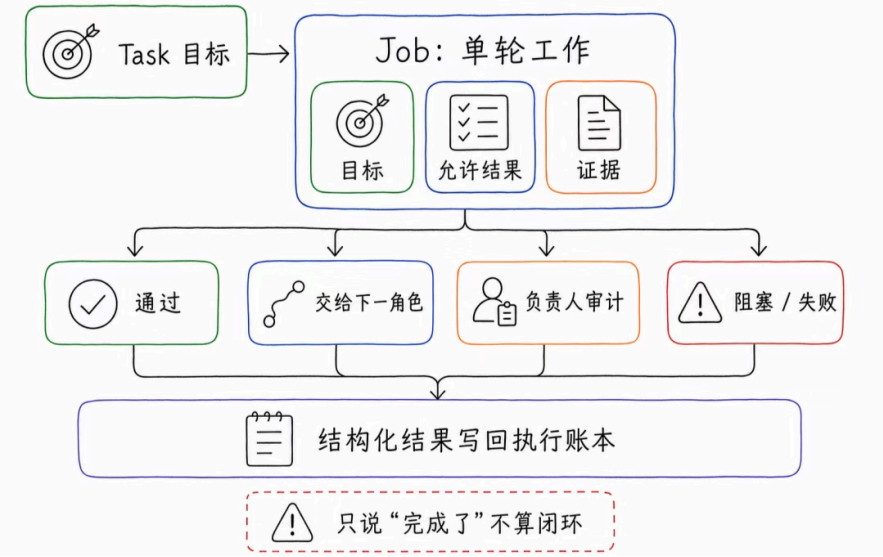
今年 3 月份，团队在讨论塔罗下一步时就提出过一个初步判断：补齐视觉上下文（d2c）能力以后，前端 AI Coding 将进入“目标牵引—监督—循环—节点执行”的全链路，人只负责目标确认和 Review。

当前实践仍然沿着这一方向推进。当时尚未被充分认识的难点在于，Prompt 中的重复提醒无法形成目标牵引；所有执行对象和状态迁移都必须关联同一个目标。

## 04 Agent 是能力提供者，Job 才是最小执行单元

塔罗将 **Job** 作为最核心的执行抽象。

Agent 表示一种能力和工程责任：前端实现、视觉修正、代码审查、用户路径验收或者过程仲裁。Job 表示这个角色当前要完成的一轮具体工作。




一个可以进入自动循环的 Job，必须明确四件事：

1. 这一轮要解决什么问题；
2. 可以用哪些结构化结果结束；
3. 结束时必须留下什么证据；
4. 不同结果分别流向哪里。

例如，前端开发 Agent 完成一次 UI 可见改动后，结论不能只有一段自然语言总结，它可能以下任意结果：

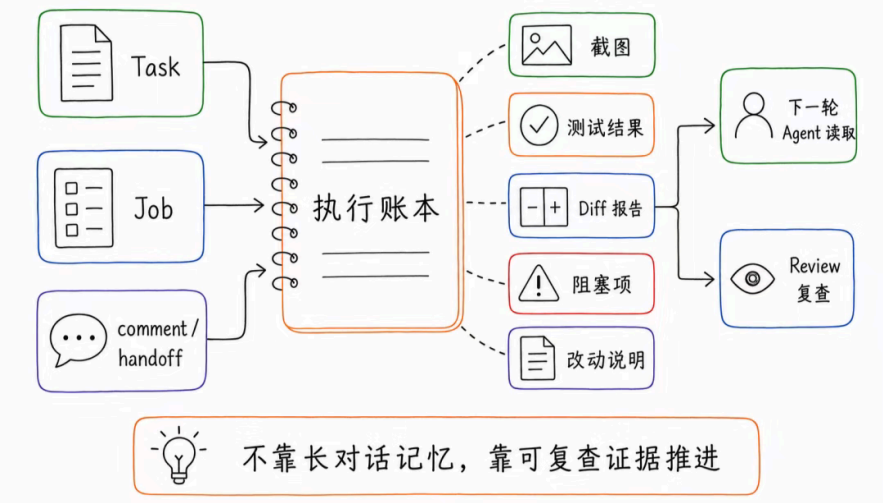
1. 结构实现完成，交给视觉 Agent
2. 无需视觉验证，并给出理由
3. 缺少接口条件，交给负责人仲裁
4. 当前方案失败
5. 任务被新要求中断

不同结果对应不同的下一步，系统使用结构化结果推进状态，避免从大段模型回复中推测完成意图，如果 Agent 执行结束，却没有给出当前 Job 接受的结果，系统不会把它当成完成，而会标记为未闭环，进入审计或仲裁。

这一约束直接决定单轮工作能否进入系统循环：Agent 声明完成不代表当前工作闭环；只有目标、证据和出口同时成立，闭环才能真正形成。

以 Job 为最小执行单元，可以把失败限制在当前一轮。视觉修正失败时，流程回到前端 Agent 补充结构；Review 证据不足时，流程回到对应角色补齐证据。Task 从受影响的阶段继续推进，不需要整体重启。

Loop 要运行得足够久，就不能依赖某一个 Agent 记住所有事情，塔罗会把 Task、Job、结构化结果、Comment、Handoff、截图、测试结果、Diff 报告和阻塞项写入执行账本。




执行账本需要同时容纳事实证据与阶段判断。文件改动、运行命令、验证 URL、Mock 场景、截图、Diff 报告和测试结果属于可复查事实；方案理由、剩余风险、后续关注点和责任角色则用于支撑下一轮决策。

前者进入执行账本，后者通过 Comment 和 Handoff 交接。下一轮 Agent 不需要重读整段历史，只需要读取当前 Task、当前 Job、最近一次交接和必要证据。

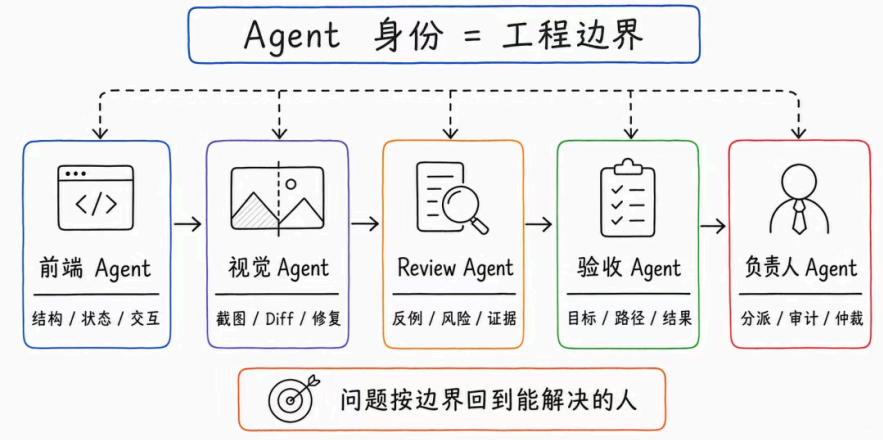
塔罗为 Review Agent 设置了一条关键规则：**Handoff 只能作为线索，不能作为事实本身**，例如：前端开发 Agent 说“已经验证所有状态”，Review Agent 不能直接相信这句话，它需要查看对应的 Mock、截图、测试结果或者实际代码 Diff。

这是多 Agent 系统[Q]中的典型风险：如果 Agent 之间只相互读取总结，它们可能形成彼此强化、却缺少真实证据的结论。执行账本将可复查事实作为协作依据。

## 06 多 Agent：协作可以自由，状态必须受控

塔罗依据工程责任划分 Agent 角色，如果几个 Agent 使用相似的上下文、做相似的判断、最后相互复述，多 Agent 只会带来更多调用成本和上下文噪声。

拆分角色只在一种情况下有价值：**不同角色对应稳定的工程责任，并且它们对“完成”的判断标准不同**。




前端 Agent 负责组件结构、业务状态、数据流和交互入口；视觉 Agent 负责截图、Diff 和可见结果修正；Review Agent 默认寻找反例和风险；验收 Agent 按用户目标和用户路径判断是否可交付；负责人 Agent 处理分派、冲突和无法自动解决的异常。

角色协作的价值取决于问题能否沿工程边界流向具备处理能力的 Agent。

例如：元素的位置偏差由视觉 Agent 修复；状态分支缺失需要退回前端 Agent；设计稿与需求发生冲突时转交负责人或用户；Review Agent 发现结论缺少证据时，则将 Job 退回补充证据。每类问题都有明确出口，不需要重新执行一轮泛化审查。

角色边界解决了“由谁处理”，但还没有解决“协作如何可靠发生”。塔罗将模型执行与业务工作分开管理：一次模型调用可以被中断、恢复或重试，其结束不会直接改变 Job 的完成状态。这样可以避免一段回复结束、一条评论出现或某个 Agent 声明完成时，系统误判任务已经推进。

协作消息也具有不同语义。Comment 用于沉淀共享事实，不一定触发执行；Mention 将明确问题投递给指定 Agent；Handoff 保存已有成果、证据和风险。流程只接受“完成、阻塞、失败”等结构化工作结果，普通消息不会直接改变任务状态。

***A 发起 @mention***

- ***→ 系统为 B 创建 Job 并附带上下文***
- ***→ B 执行并提交结构化结果***
- ***→ 系统保存交付物和回执***
- ***→ 流程根据结果继续路由***

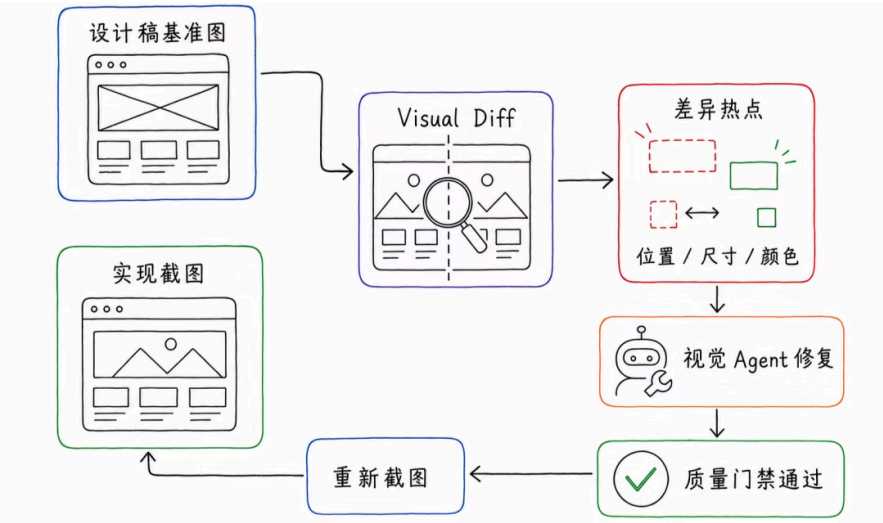
Agent 忙碌时，新信息不会被直接丢弃，也不会无条件中断当前执行。系统根据运行状态将信息送入当前执行、延迟到下一轮或进入任务队列，并在安全边界恢复工作。

这一层协作基础设施为 Agent 沟通和任务状态采用了两套规则：沟通方式接近真实团队，状态变化则必须经过受控、可验证的协议。

# 07 前端 UI 任务中的 Pixel 小循环

在塔罗 Cockpit 的大循环中，前端 UI 任务会经过实现与视觉验证两个责任阶段。前端 Agent 负责页面结构、业务状态和交互入口；只要改动影响用户可见结果，任务就会进入视觉 Agent 的验证阶段。

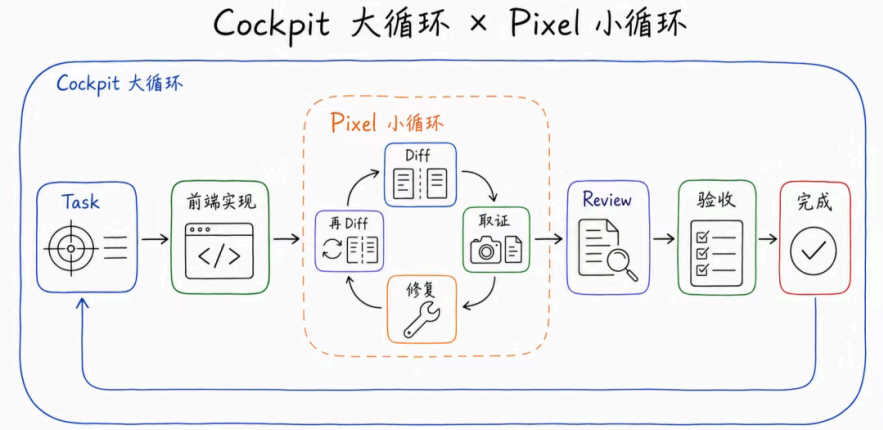
Tarot Pixel diff skill 为这个阶段提供一段完整的视觉反馈链路：




前端 Agent 完成 UI 实现后，系统分别历经以下几个步骤

1. 通过 Mock 固定目标业务状态
2. 使用 Tarot Pixel Diff 对比设计稿与实现截图
3. 根据差异进行设计侧与实现侧取证
4. 视觉 Agent 按根因修复
5. 使用相同状态和范围重新对比
6. 视觉通过，或退回前端 Agent / 负责人处理

**这个过程构成了塔罗 Cockpit 大循环内部的一个小循环**：



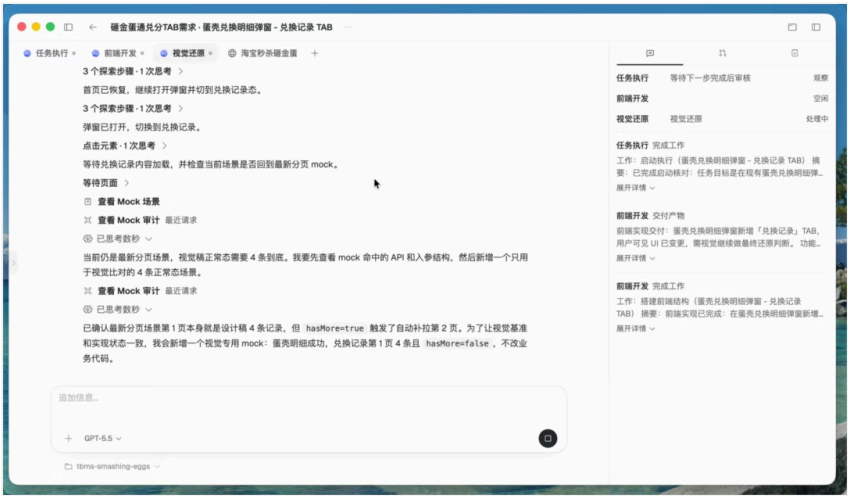

有效比较依赖稳定的业务需求背景输入，同一个页面可能包含多种状态，视觉 Agent 会通过 Mock 固定场景，再将实现结果与对应设计状态比较。Mask 仅处理商品图、用户头像等无法控制的运行时内容，固定文案、装饰和业务状态仍然参与验收。

Pixel 生成的设计稿、实现图和 Diff 图用于定位缺失、多余、错位或尺寸不一致的区域。确定修复方式时，视觉 Agent 还需要进行双边取证：设计侧从 Pixel API 获取节点尺寸、相对位置和样式，实现侧通过浏览器读取实际 DOM 的位置与计算样式。两侧数据共同构成修改依据，避免根据截图目测参数。

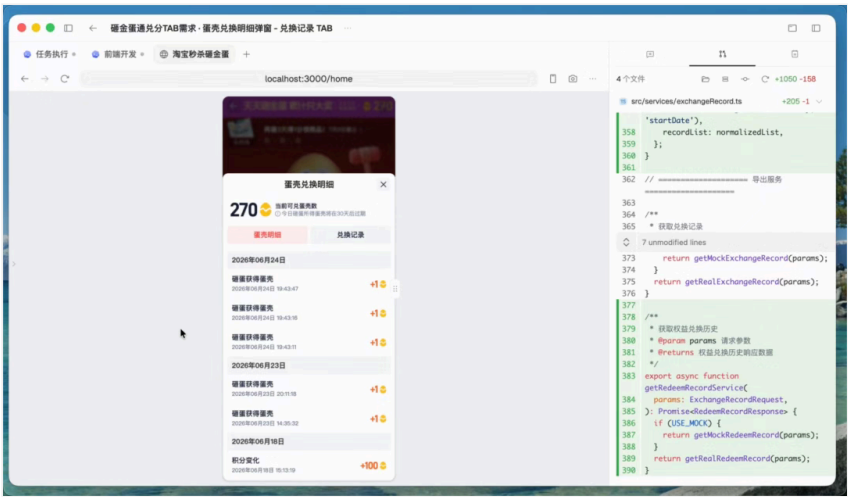
代码修改完成后，视觉 Agent 会在相同 Mock、视口和对比范围下重新执行 Diff。通过的视觉证据进入 Review；状态、结构或接口缺失会使任务退回前端 Agent；多轮修复仍无法收敛时，则转入负责人或人工仲裁。Pixel 小循环由此成为 UI Job 的内部执行过程，并通过结构化结果重新接入 Cockpit 主流程。

## 08 实践示例：多状态 Mock 与视觉修正过程

### 编码与状态 Mock 验证

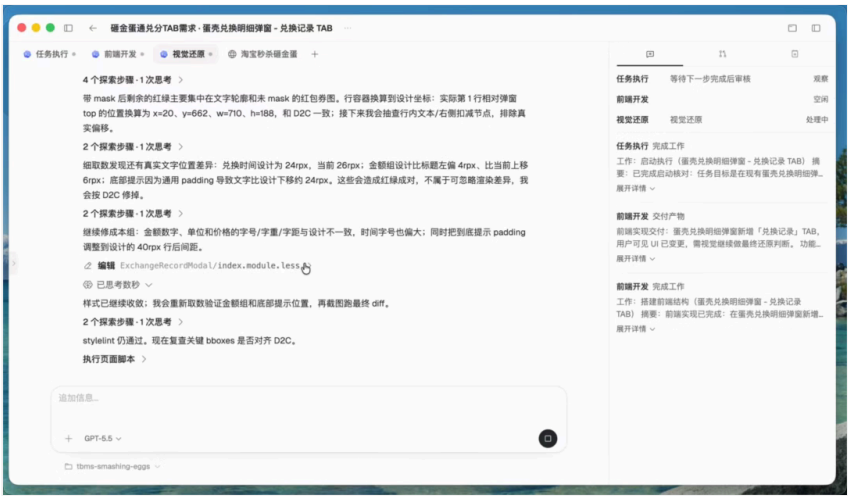




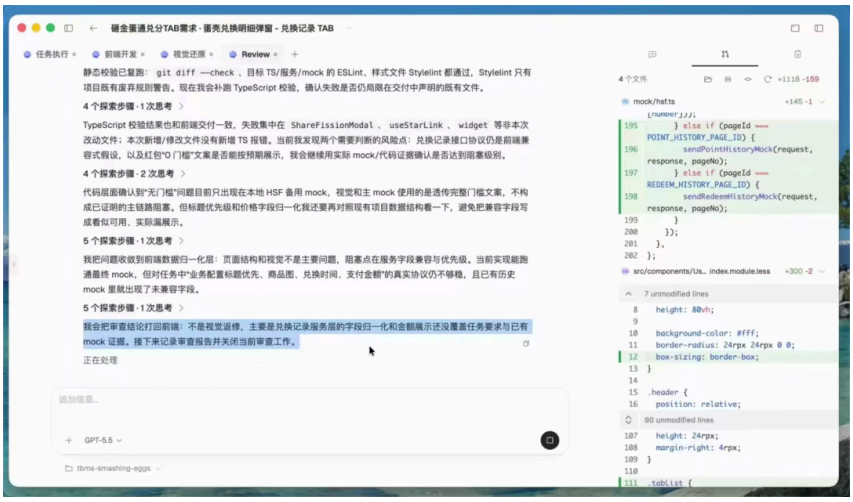



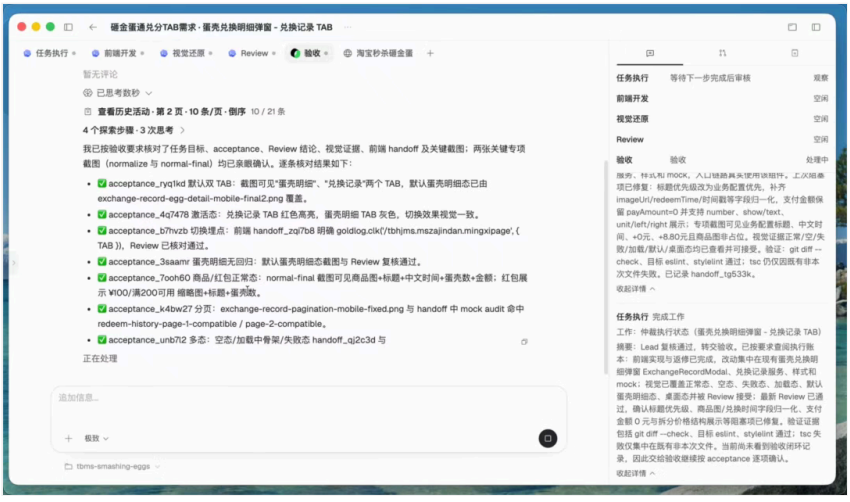

## 视觉 Diff 与修正过程





## Review 打回 & 最终验收





09 总结

塔罗对 AI Coding 的探索逐步从 Prompt 扩展到 Context，再进入 Loop。Prompt 处理单次需求的准确表达，Context 负责在正确阶段提供业务、代码和视觉信息；Loop 进一步处理复杂任务中的执行偏差，使其转化为下一轮输入，并在无法收敛时主动停止。

塔罗以 Task 保存目标，以 Job 约束单轮执行，再通过执行账本、角色路由和 Pixel 小循环持续吸收执行结果。验证门禁决定下一步操作，重试上限和人工仲裁负责终止无法收敛的路径。这些机制共同承接了过去由人完成的目标记忆、结果观察、差异判断、返修推动和结束决策。

下一步，塔罗会把当前聚焦于 Coding Agent 的执行闭环继续向研发流程上下游扩展，最终希望形成一套端到端的研发闭环：**输入一个需求，交付一份经过实现、验证和审查、可以进入发布的代码**。 在这条链路中，Agent 不只是完成某个编码节点，而是围绕同一个目标持续推进整个研发过程；人则主要负责目标确认、关键边界决策和异常仲裁。




本文作者木偶，来自淘天集团–用户场景营销技术团队。一支专注于探索AI等前沿技术与营销业务技术的融合，深度结合用户场景与营销业务的技术团队。依托大淘宝丰富的用户生态和多元化的消费场景，致力于通过技术创新提升用户体验，优化个性化营销能力，助力业务持续增长。通过AI驱动的精准推荐、场景化表达与动态策略调控，我们为用户创造更自然、更智能的购物旅程，为营销业务提供高效、敏捷的技术支撑，助力淘宝构建以用户为中心的全域营销技术体系。我们坚信技术是连接用户与价值的桥梁，持续探索创新边界，让营销更懂用户，用技术点亮每一个关键用户体验瞬间。

**¤ 拓展阅读 ¤**

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
966篇原创内容
公众号

阅读 3740

## 留言

写留言

还没有留言，来写首评